Enterprise ETL Design – Health Insurance Bulk Onboarding

(Oracle Database + Python Orchestration)

1□. Excel File Design (Single File, Single Sheet)

File name (example):
CLIENT_INSURANCE_UPLOAD.xlsx

Sheet name:
EMPLOYEES

Excel Columns (Business + Compliance Ready)

Excel Columns (Business + Compliance Ready)

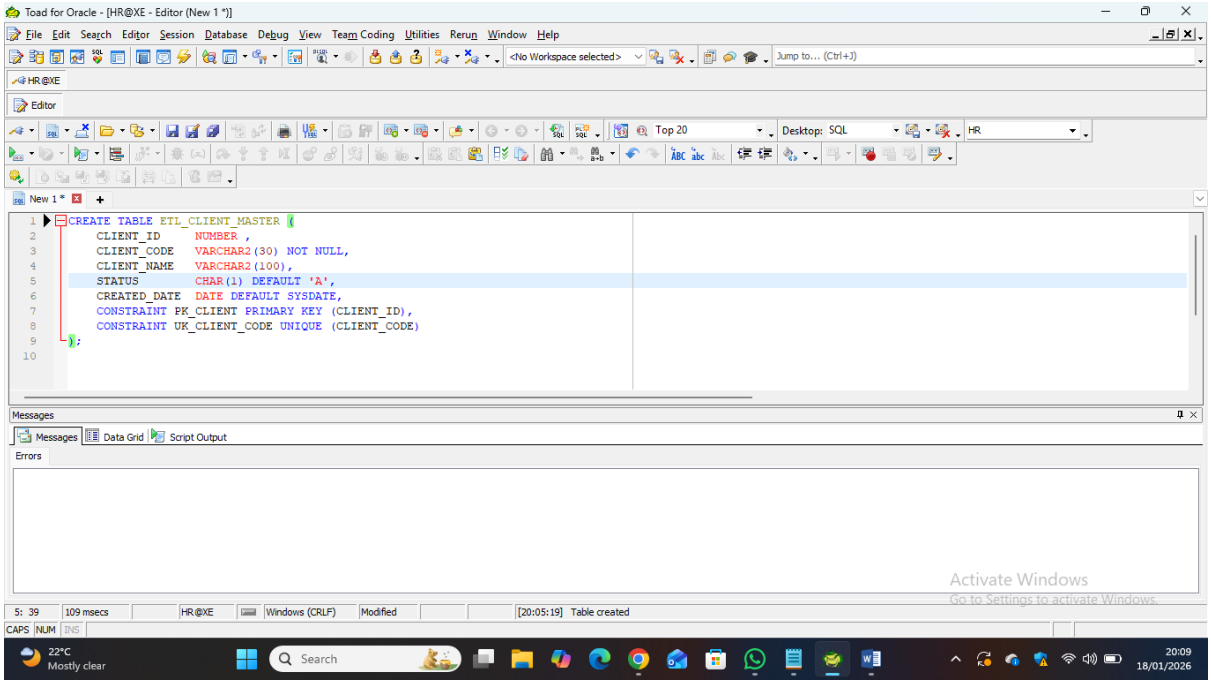
Column Name	Description
CLIENT_CODE	Unique client identifier
EMP_CODE	Employee code provided by client
EMP_NAME	Employee full name
CNIC	National ID (primary identity)
PASSPORT_NO	Passport number (nullable)
DATE_OF_BIRTH	DOB
GENDER	M / F
MOBILE_NO	Contact
EMAIL	Optional
JOIN_DATE	Employee joining date
INSURANCE_START_DATE	Policy start
INSURANCE_END_DATE	Nullable (active if null)

🔹 This structure supports **20,000 bulk rows + future incremental loads**

2. Core Database Tables (Final Design)

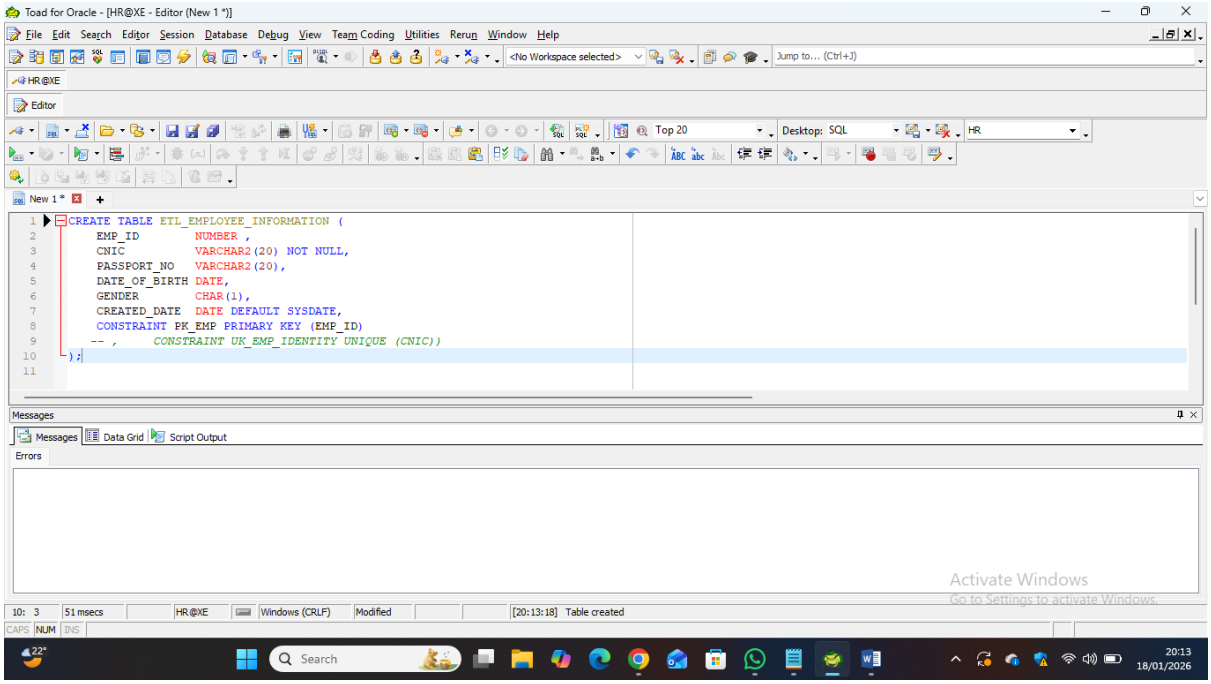
CLIENT_MASTER (Master)

```
CREATE TABLE ETL_CLIENT_MASTER (
   CLIENT_ID     NUMBER ,
   CLIENT_CODE   VARCHAR2(30) NOT NULL,
   CLIENT_NAME   VARCHAR2(100),
   STATUS        CHAR(1) DEFAULT 'A',
   CREATED_DATE  DATE DEFAULT SYSDATE,
   CONSTRAINT PK_CLIENT PRIMARY KEY (CLIENT_ID),
   CONSTRAINT UK_CLIENT_CODE UNIQUE (CLIENT_CODE)
);
```



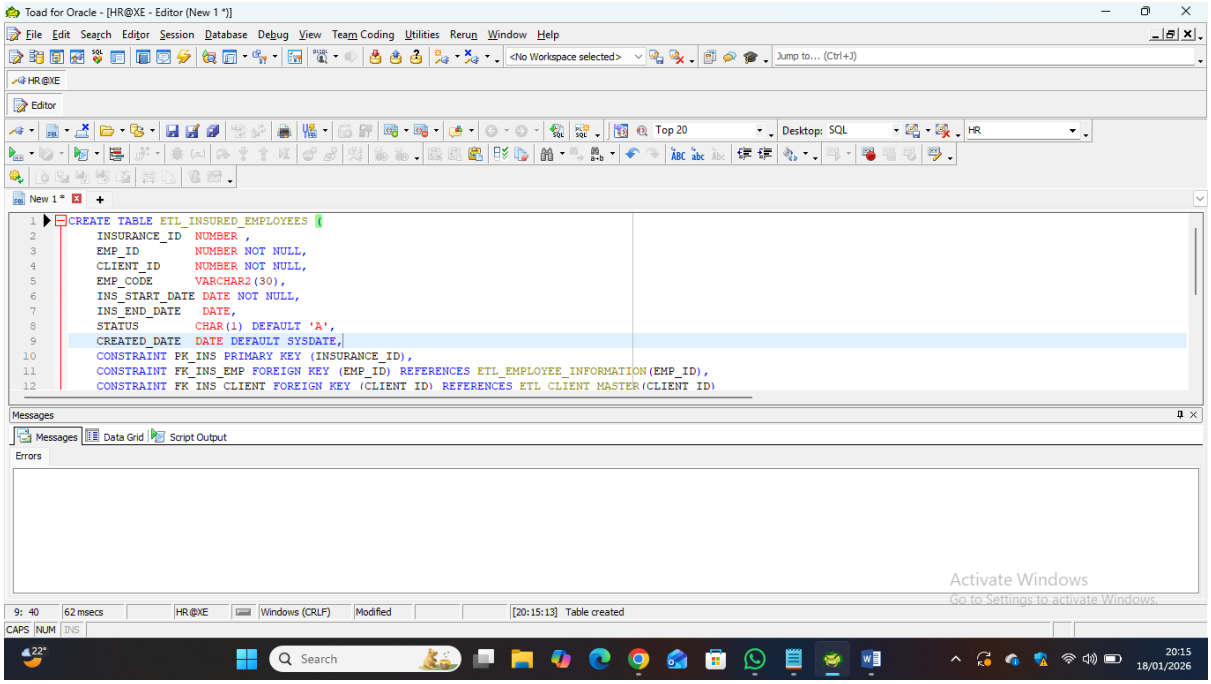
EMPLOYEE_INFORMATION (Master – Global Identity)

```
CREATE TABLE ETL_EMPLOYEE_INFORMATION (
   EMP_ID        NUMBER ,
   CNIC          VARCHAR2(20) NOT NULL,
   PASSPORT_NO   VARCHAR2(20),
   DATE_OF_BIRTH DATE,
   GENDER        CHAR(1),
   CREATED_DATE  DATE DEFAULT SYSDATE,
   CONSTRAINT PK_EMP PRIMARY KEY (EMP_ID)
  -- ,     CONSTRAINT UK_EMP_IDENTITY UNIQUE (CNIC))
);
```



INSURED_EMPLOYEES (Transaction Table)

```
CREATE TABLE ETL_INSURED_EMPLOYEES (
   INSURANCE_ID  NUMBER ,
   EMP_ID        NUMBER NOT NULL,
   CLIENT_ID     NUMBER NOT NULL,
   EMP_CODE      VARCHAR2(30),
   INS_START_DATE DATE NOT NULL,
   INS_END_DATE   DATE,
   STATUS        CHAR(1) DEFAULT 'A',
   CREATED_DATE  DATE DEFAULT SYSDATE,
   CONSTRAINT PK_INS PRIMARY KEY (INSURANCE_ID),
   CONSTRAINT FK_INS_EMP FOREIGN KEY (EMP_ID) REFERENCES
ETL_EMPLOYEE_INFORMATION(EMP_ID),
   CONSTRAINT FK_INS_CLIENT FOREIGN KEY (CLIENT_ID) REFERENCES
ETL_CLIENT_MASTER(CLIENT_ID)
);
```



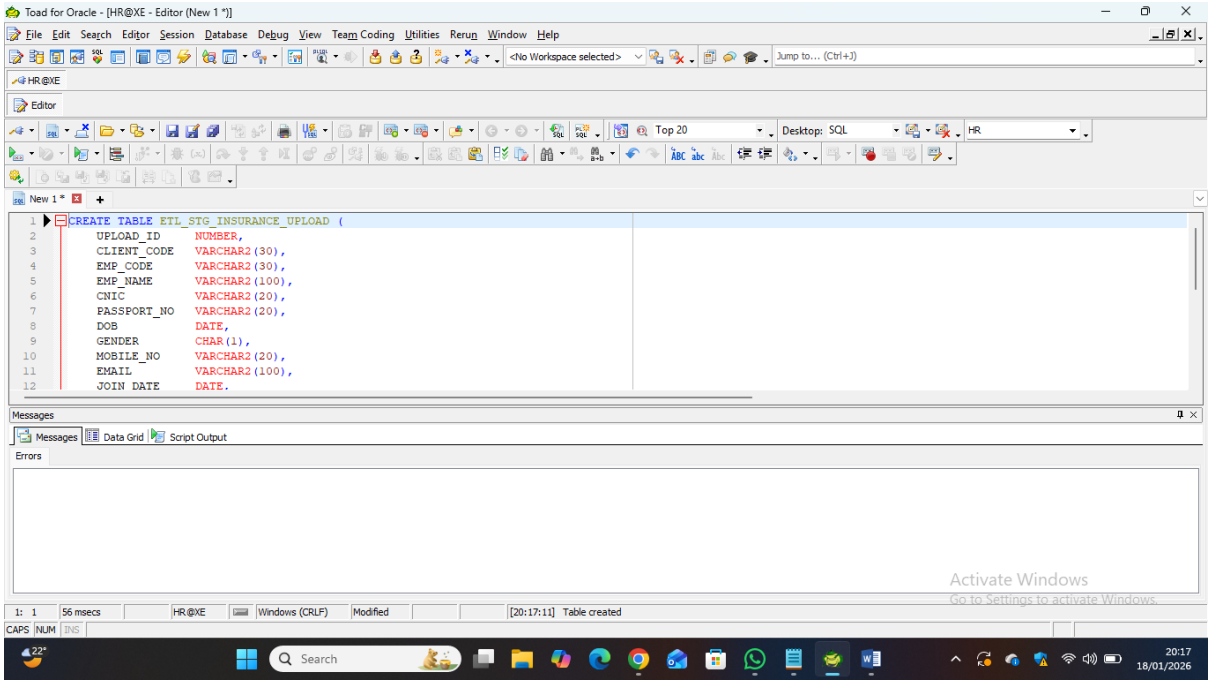
Business Rule (One Active Insurance at a Time)

Enforced via **PL/SQL validation**, not hard constraint:

-- STATUS = 'A' and INS_END_DATE IS NULL means ACTIVE

3. STAGING TABLE (Mediator – Mandatory)

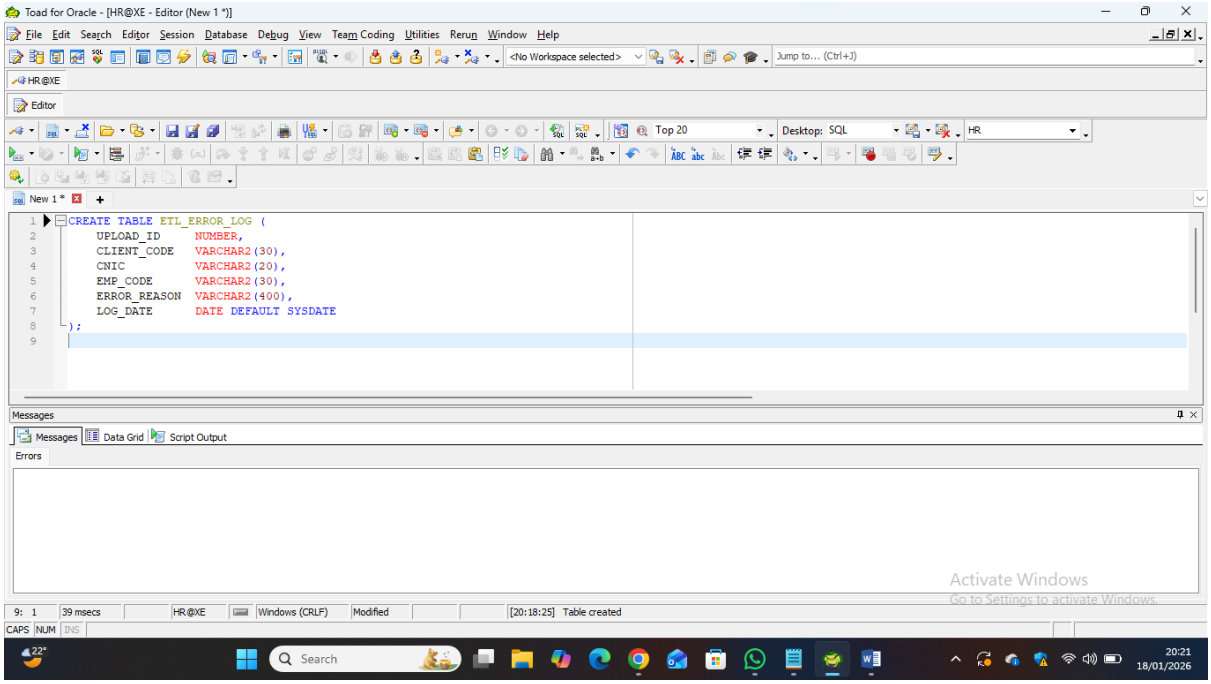
```
CREATE TABLE ETL_STG_INSURANCE_UPLOAD (
   UPLOAD_ID     NUMBER,
   CLIENT_CODE   VARCHAR2(30),
   EMP_CODE      VARCHAR2(30),
   EMP_NAME      VARCHAR2(100),
   CNIC          VARCHAR2(20),
   PASSPORT_NO   VARCHAR2(20),
   DOB           DATE,
   GENDER        CHAR(1),
   MOBILE_NO     VARCHAR2(20),
   EMAIL         VARCHAR2(100),
   JOIN_DATE     DATE,
   INS_START_DATE DATE,
   INS_END_DATE   DATE
);
```



- No constraints
- Bulk-friendly
- Re-runnable

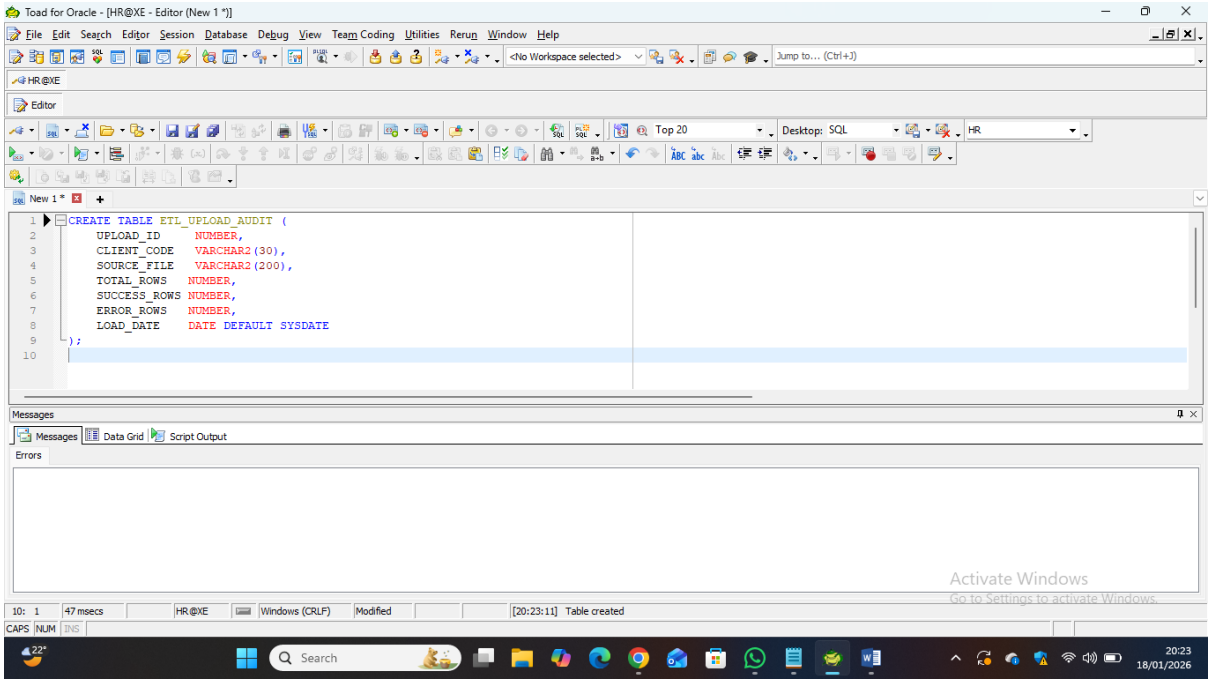
4. ERROR LOG TABLE (Row-Level Control)

```
CREATE TABLE ETL_ERROR_LOG (
   UPLOAD_ID     NUMBER,
   CLIENT_CODE   VARCHAR2(30),
   CNIC          VARCHAR2(20),
   EMP_CODE      VARCHAR2(30),
   ERROR_REASON  VARCHAR2(400),
   LOG_DATE      DATE DEFAULT SYSDATE
);
```



5. AUDIT / CONTROL TABLE

```
CREATE TABLE ETL_UPLOAD_AUDIT (
   UPLOAD_ID     NUMBER,
   CLIENT_CODE   VARCHAR2(30),
   SOURCE_FILE   VARCHAR2(200),
   TOTAL_ROWS    NUMBER,
   SUCCESS_ROWS NUMBER,
   ERROR_ROWS    NUMBER,
   LOAD_DATE     DATE DEFAULT SYSDATE
);
```



6. Validation Rules (As You Defined – Locked)

File-Level (Python)

- Duplicate CNIC in same Excel → ERROR
- Mandatory column missing → ERROR

Database-Level (PL/SQL)

1. CNIC exists + last insurance **ACTIVE** → ERROR
2. CNIC exists + last insurance **INACTIVE** → ALLOW
3. New CNIC → ALLOW

7. Python Responsibility (Controller)

- Read Excel
- Generate UPLOAD_ID
- Bulk insert into STG_INSURANCE_UPLOAD
- Call PL/SQL procedure
- Read audit results

Python **does not decide business rules**
Oracle **does**

8. PL/SQL Responsibility (Authority)

- Validate staging data
- Insert CLIENT (if new)
- Insert EMPLOYEE_INFORMATION (if new)
- Insert INSURED_EMPLOYEES
- Log errors
- Update audit

9. Why This Design Is Perfect

✓ Bulk-safe
✓ Incremental-ready
✓ Fully auditable
✓ Insurance-compliant
✓ No ETL tools
✓ Pure Oracle + Python
✓ Handles real-world employee movement

Full ETL Procedure (Dynamic IDs, Complete Logic)

```
CREATE OR REPLACE PROCEDURE HR.ETL_PRC_INSURANCE_LOAD (
   p_upload_id   IN NUMBER,
   p_client_code IN VARCHAR2,
   p_source_file IN VARCHAR2
) IS

   v_client_id     NUMBER;
   v_emp_id        NUMBER;
   v_ins_id        NUMBER;
   v_active_cnt    NUMBER;

   v_total_rows    NUMBER;
   v_error_rows    NUMBER;
   v_success_rows  NUMBER;

BEGIN
   /* ============================================================
     1. Resolve / Create CLIENT
     ============================================================ */
   BEGIN
      SELECT client_id
      INTO v_client_id
      FROM etl_client_master
      WHERE client_code = p_client_code;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         SELECT NVL(MAX(client_id),0)+1
         INTO v_client_id
         FROM etl_client_master;

         INSERT INTO etl_client_master (client_id, client_code, client_name)
         SELECT DISTINCT v_client_id, client_code, NULL
         FROM etl_stg_insurance_upload
         WHERE upload_id = p_upload_id
           AND client_code = p_client_code;
   END;

   /* ============================================================
     2. Process Staging Rows
     ============================================================ */
   FOR r IN (
      SELECT *
      FROM etl_stg_insurance_upload
      WHERE upload_id = p_upload_id
        AND client_code = p_client_code
   ) LOOP

      /* ---------------------------------------------
```

```
   2.1 Check ACTIVE insurance for same CNIC
   ---------------------------------------------- */
SELECT COUNT(*)
INTO v_active_cnt
FROM etl_insured_employees ie
JOIN etl_employee_information ei
  ON ie.emp_id = ei.emp_id
WHERE ei.cnic = r.cnic
  AND ie.status = 'A'
  AND ie.ins_end_date IS NULL;

IF v_active_cnt > 0 THEN
    INSERT INTO etl_error_log
    (upload_id, client_code, cnic, emp_code, error_reason)
    VALUES
    (p_upload_id, p_client_code, r.cnic, r.emp_code,
     'Employee already ACTIVE with another client');
    CONTINUE;
END IF;

/* ----------------------------------------------
   2.2 Resolve / Create EMPLOYEE
   ---------------------------------------------- */
BEGIN
    SELECT emp_id
    INTO v_emp_id
    FROM etl_employee_information
    WHERE cnic = r.cnic;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        SELECT NVL(MAX(emp_id),0)+1
        INTO v_emp_id
        FROM etl_employee_information;

        INSERT INTO etl_employee_information
        (emp_id, cnic, passport_no, date_of_birth, gender)
        VALUES
        (v_emp_id, r.cnic, r.passport_no, r.dob, r.gender);
END;

/* ----------------------------------------------
   2.3 Insert Insurance Transaction
   ---------------------------------------------- */
SELECT NVL(MAX(insurance_id),0)+1
INTO v_ins_id
FROM etl_insured_employees;

INSERT INTO etl_insured_employees
(insurance_id, emp_id, client_id, emp_code,
 ins_start_date, ins_end_date, status)
```

```
        VALUES
        (v_ins_id, v_emp_id, v_client_id, r.emp_code,
         r.ins_start_date, r.ins_end_date, 'A');

    END LOOP;

    /* ============================================================
       3. Audit Summary – Fixed to avoid ORA-00937
       ============================================================ */
    -- Compute totals in variables
    SELECT COUNT(*)
    INTO v_total_rows
    FROM etl_stg_insurance_upload
    WHERE upload_id = p_upload_id;

    SELECT COUNT(*)
    INTO v_error_rows
    FROM etl_error_log
    WHERE upload_id = p_upload_id;

    v_success_rows := v_total_rows - v_error_rows;

    -- Insert into audit table
    INSERT INTO etl_upload_audit
    (upload_id, client_code, source_file, total_rows, success_rows, error_rows)
    VALUES
    (p_upload_id, p_client_code, p_source_file,
     v_total_rows, v_success_rows, v_error_rows);

    COMMIT;

END;
/
```

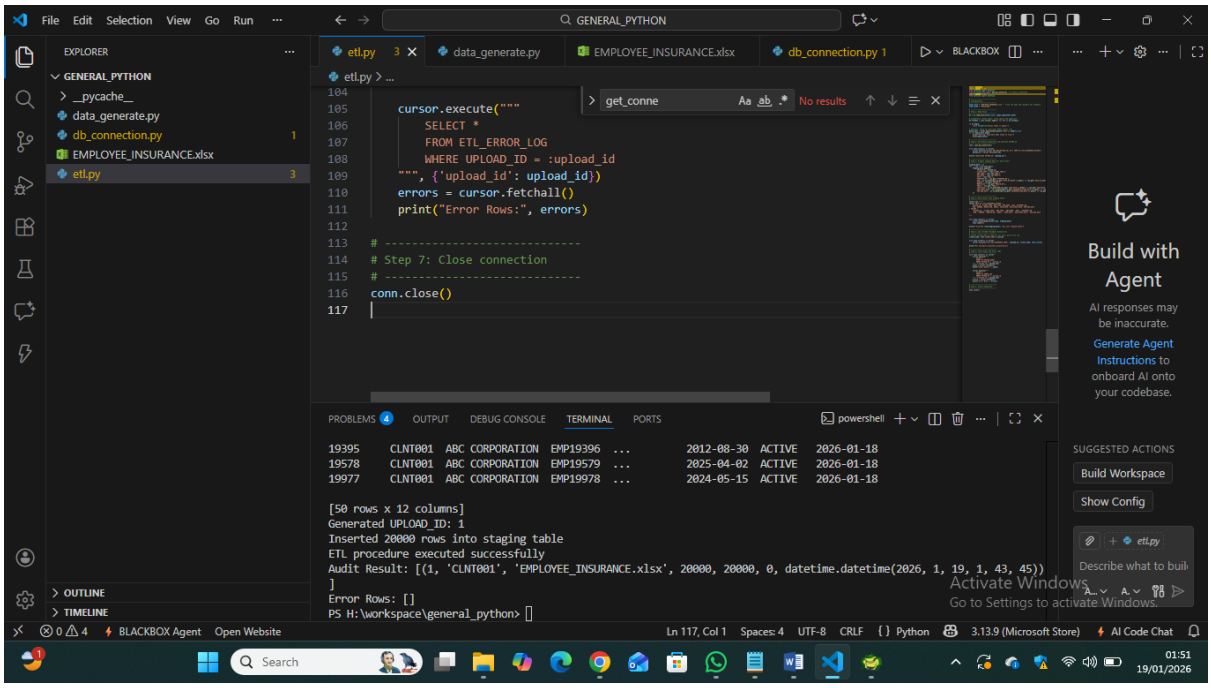
Python Code for ETL

```
import oracledb
import pandas as pd
from datetime import datetime
from db_connection import get_db_connection  # ✅ Oracle connection
import pandas as pd
from datetime import datetime

# ----------------------------
# Configuration
# ----------------------------
EXCEL_FILE = "EMPLOYEE_INSURANCE.xlsx"   # You can make this dynamic per schedule
SHEET_NAME = "EMPLOYEES"
```

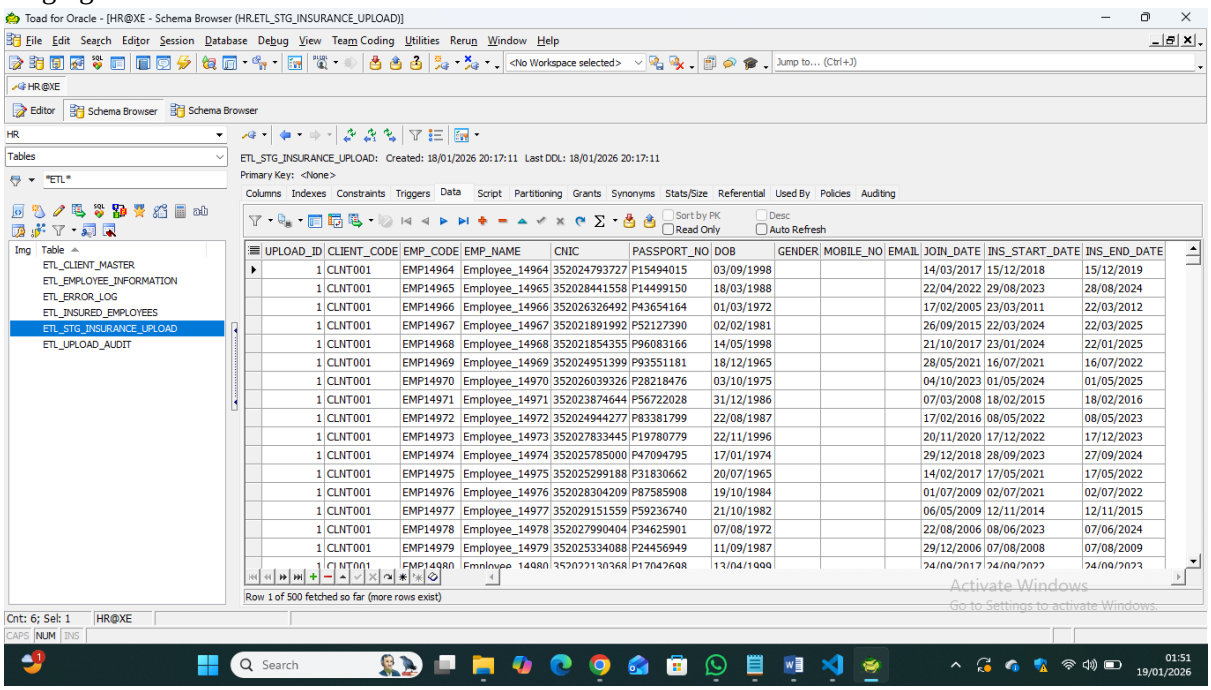
```
# ------------------------------
# Step 1: Read Excel
# ------------------------------
df = pd.read_excel(EXCEL_FILE, sheet_name=SHEET_NAME)

# Standardize column names: strip spaces and uppercase
df.columns = [col.strip().upper() for col in df.columns]

if df.empty:
    raise ValueError("Excel sheet is empty!")

# Optional: Check for duplicate CNICs within file
duplicates = df[df.duplicated(subset=['CNIC'], keep=False)]
if not duplicates.empty:
    print("Warning: Duplicate CNIC found in file:")
    print(duplicates)

# ------------------------------
# Step 2: Get Oracle connection and generate UPLOAD_ID
# ------------------------------
conn = get_db_connection()

with conn.cursor() as cursor:
    cursor.execute("SELECT NVL(MAX(UPLOAD_ID),0)+1 FROM
ETL_STG_INSURANCE_UPLOAD")
    upload_id = cursor.fetchone()[0]

print(f"Generated UPLOAD_ID: {upload_id}")

# ------------------------------
# Step 3: Prepare staging data for bulk insert
# ------------------------------
staging_data = []
for _, row in df.iterrows():
    staging_data.append({
        'UPLOAD_ID': upload_id,
        'CLIENT_CODE': row['CLIENT_CODE'],
        'EMP_CODE': row['EMP_CODE'],
        'EMP_NAME': row['EMP_NAME'],
        'CNIC': row['CNIC'],
        'PASSPORT_NO': row.get('PASSPORT_NO'),
        'DOB': pd.to_datetime(row.get('DATE_OF_BIRTH')).date() if row.get('DATE_OF_BIRTH')
else None,
        'GENDER': row.get('GENDER'),
        'MOBILE_NO': row.get('MOBILE_NO'),
        'EMAIL': row.get('EMAIL'),
        'JOIN_DATE': pd.to_datetime(row.get('JOIN_DATE')).date() if row.get('JOIN_DATE') else
None,
        'INS_START_DATE': pd.to_datetime(row.get('INSURANCE_START_DATE')).date() if
row.get('INSURANCE_START_DATE') else None,
```

```
        'INS_END_DATE': pd.to_datetime(row.get('INSURANCE_END_DATE')).date() if
row.get('INSURANCE_END_DATE') else None
    })

# ------------------------------
# Step 4: Bulk Insert into staging table
# ------------------------------
insert_sql = """
INSERT INTO ETL_STG_INSURANCE_UPLOAD (
    UPLOAD_ID, CLIENT_CODE, EMP_CODE, EMP_NAME, CNIC, PASSPORT_NO,
    DOB, GENDER, MOBILE_NO, EMAIL, JOIN_DATE, INS_START_DATE, INS_END_DATE
) VALUES (
    :UPLOAD_ID, :CLIENT_CODE, :EMP_CODE, :EMP_NAME, :CNIC, :PASSPORT_NO,
    :DOB, :GENDER, :MOBILE_NO, :EMAIL, :JOIN_DATE, :INS_START_DATE,
:INS_END_DATE
)
"""

with conn.cursor() as cursor:
    cursor.executemany(insert_sql, staging_data)
    conn.commit()

print(f"Inserted {len(staging_data)} rows into staging table")

# ------------------------------
# Step 5: Call PL/SQL Procedure dynamically
# ------------------------------
# Assume client_code is same for all rows; pick first row
client_code = df['CLIENT_CODE'].iloc[0]

with conn.cursor() as cursor:
    cursor.callproc("ETL_PRC_INSURANCE_LOAD", [upload_id, client_code, EXCEL_FILE])

print("ETL procedure executed successfully")

# ------------------------------
# Step 6: Fetch Audit and Error logs
# ------------------------------
with conn.cursor() as cursor:
    cursor.execute("""
        SELECT *
        FROM ETL_UPLOAD_AUDIT
        WHERE UPLOAD_ID = :upload_id
    """, {'upload_id': upload_id})
    audit = cursor.fetchall()
    print("Audit Result:", audit)

    cursor.execute("""
        SELECT *
        FROM ETL_ERROR_LOG
        WHERE UPLOAD_ID = :upload_id
```

```
    """, {'upload_id': upload_id})
    errors = cursor.fetchall()
    print("Error Rows:", errors)

# ----------------------------
# Step 7: Close connection
# ----------------------------
conn.close()
```

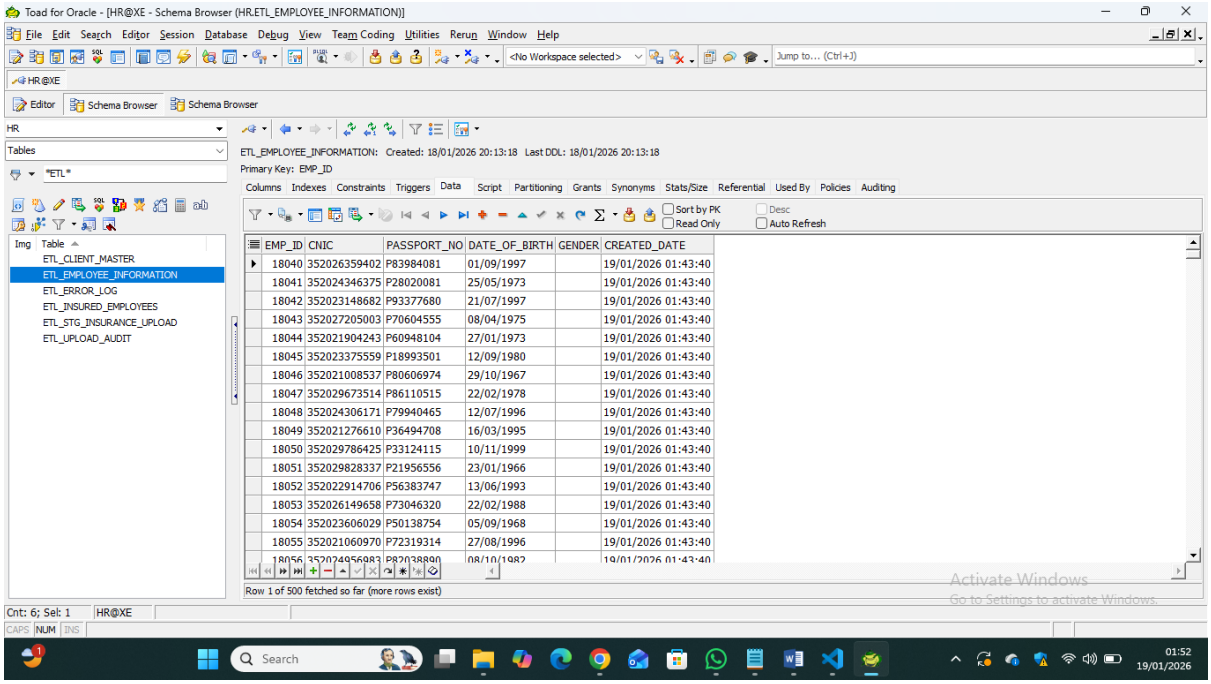


Staging Table



UPLOAD_ID	CLIENT_CODE	EMP_CODE	EMP_NAME	CNIC	PASSPORT_NO	DOB	GENDER	MOBILE_NO	EMAIL	JOIN_DATE	INS_START_DATE	INS_END_DATE
1	CLNT001	EMP14964	Employee_14964	352024793727	P15494015	03/09/1998				14/03/2017	15/12/2018	15/12/2019
1	CLNT001	EMP14965	Employee_14965	352028441558	P14499150	18/03/1988				22/04/2022	29/08/2023	28/08/2024
1	CLNT001	EMP14966	Employee_14966	352026326492	P43654164	01/03/1972				17/02/2005	23/03/2011	22/03/2012
1	CLNT001	EMP14967	Employee_14967	352021891992	P52127390	02/02/1981				26/09/2015	22/03/2024	22/03/2025
1	CLNT001	EMP14968	Employee_14968	352021854355	P96083166	14/05/1998				21/10/2017	23/01/2024	22/01/2025
1	CLNT001	EMP14969	Employee_14969	352024951399	P93551181	18/12/1965				28/05/2021	16/07/2021	16/07/2022
1	CLNT001	EMP14970	Employee_14970	352026039326	P28218476	03/10/1975				04/10/2023	01/05/2024	01/05/2025
1	CLNT001	EMP14971	Employee_14971	352023874644	P56722028	31/12/1986				07/03/2008	18/02/2015	18/02/2016
1	CLNT001	EMP14972	Employee_14972	352024944277	P83381799	22/08/1987				17/02/2016	08/05/2022	08/05/2023
1	CLNT001	EMP14973	Employee_14973	352027833445	P19780779	22/11/1996				20/11/2020	17/12/2022	17/12/2023
1	CLNT001	EMP14974	Employee_14974	352025785000	P47094795	17/01/1974				29/12/2018	28/09/2023	27/09/2024
1	CLNT001	EMP14975	Employee_14975	352025299188	P31830662	20/07/1965				14/02/2017	17/05/2021	17/05/2022
1	CLNT001	EMP14976	Employee_14976	352028304209	P87585908	19/10/1984				01/07/2009	02/07/2021	02/07/2022
1	CLNT001	EMP14977	Employee_14977	352029151559	P59236740	21/10/1982				06/05/2009	12/11/2014	12/11/2015
1	CLNT001	EMP14978	Employee_14978	352027990404	P34625901	07/08/1972				22/08/2006	08/06/2023	07/06/2024
1	CLNT001	EMP14979	Employee_14979	352025334088	P24456949	11/09/1987				29/12/2006	07/08/2008	07/08/2009
1	CLNT001	EMP14980	Employee_14980	352022130368	P17042698	13/04/1999				24/09/2017	24/09/2022	24/09/2023

Final Table



EMP_ID	CNIC	PASSPORT_NO	DATE_OF_BIRTH	GENDER	CREATED_DATE
18040	352026359402	P83984081	01/09/1997		19/01/2026 01:43:40
18041	352024346375	P28020081	25/05/1973		19/01/2026 01:43:40
18042	352023148682	P93377680	21/07/1997		19/01/2026 01:43:40
18043	352027205003	P70604555	08/04/1975		19/01/2026 01:43:40
18044	352021904243	P60948104	27/01/1973		19/01/2026 01:43:40
18045	352023375559	P18993501	12/09/1980		19/01/2026 01:43:40
18046	352021008537	P80606974	29/10/1967		19/01/2026 01:43:40
18047	352029673514	P86110515	22/02/1978		19/01/2026 01:43:40
18048	352024306171	P79940465	12/07/1996		19/01/2026 01:43:40
18049	352021276610	P36494708	16/03/1995		19/01/2026 01:43:40
18050	352029786425	P33124115	10/11/1999		19/01/2026 01:43:40
18051	352029828337	P21956556	23/01/1966		19/01/2026 01:43:40
18052	352022914706	P56383747	13/06/1993		19/01/2026 01:43:40
18053	352026149658	P73046320	22/02/1988		19/01/2026 01:43:40
18054	352023606029	P50138754	05/09/1968		19/01/2026 01:43:40
18055	352021060970	P72319314	27/08/1996		19/01/2026 01:43:40
18056	352024956983	P82038880	08/10/1982		19/01/2026 01:43:40